TY B.Tech. – ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

PRACTICAL ASSIGNMENT 5 – CLUSTERING ALGORITHMS

1. Student Details

Student Name	Revati Bhosale
PRN	202401040048
Branch	Computer Engineering
Division	C
Batch	C2
Roll No.	173
Date of Submission	22/10/2026

2. Practical Details

Practical No.	5
Practical Title	Clustering Algorithms
Date Performed	25/10/2026
Faculty Name	Dr. Khushal Khairnar

3. Problem Statement

Implement clustering algorithms on a real-world dataset to identify meaningful groups within the data. Evaluate the quality of the clustering results using appropriate metrics such as the silhouette score, the Davies-Bouldin index, or visual inspection.

The K-Means, Hierarchical (Agglomerative) and DBSCAN algorithms, together with a Gaussian Mixture Model, are applied to the Wine dataset – the results of a chemical analysis of 178 wines with 13 measured properties each. The class labels are ignored while clustering, so the task is to discover natural groups of similar wines without supervision.

4. Objectives

1. To implement K-Means, Hierarchical, DBSCAN and Gaussian Mixture clustering on a real-world dataset after proper pre-processing (feature scaling).
2. To select a suitable number of clusters using the elbow method and the silhouette score.
3. To evaluate the clusters using the silhouette score, Davies-Bouldin index, Calinski-Harabasz index and visual inspection (PCA plots, dendrogram).
4. To interpret the discovered groups and compare the algorithms.

5. Expected Learning Outcomes

After completing this practical, the student will be able to:

1. Explain how K-Means, hierarchical clustering, DBSCAN and Gaussian Mixture models form clusters.
2. Pre-process data and apply clustering algorithms in Python using scikit-learn.
3. Choose the number of clusters and the algorithm parameters (k, eps, min_samples) with justification.
4. Evaluate and interpret clustering quality with internal metrics and visualisations, and describe the meaning of the groups found.

6. Theory / Concept

Clustering: an unsupervised learning task that groups data points so that points in the same cluster are more similar to each other than to points in other clusters. No class labels are used.

K-Means: partitions the data into k clusters. It repeatedly assigns every point to the nearest centroid and then recomputes each centroid as the mean of its points, minimising the within-cluster sum of squares (inertia). It is fast but needs k in advance, assumes roughly spherical clusters and is sensitive to outliers.

Hierarchical (Agglomerative) clustering: starts with every point as its own cluster and repeatedly merges the two closest clusters. Ward linkage merges the pair that gives the smallest increase in within-cluster variance. The result is a dendrogram, which can be cut at any level to get the required number of clusters.

DBSCAN: density-based clustering controlled by eps (neighbourhood radius) and min_samples. Points with at least min_samples neighbours within eps are core points; clusters grow from connected core points and points that belong to no dense region are labelled as noise. It finds arbitrary-shaped clusters and does not need k.

Gaussian Mixture Model (GMM): assumes the data is generated from a mixture of Gaussian distributions and fits it with the Expectation-Maximisation algorithm. It gives soft (probabilistic) cluster membership and can model elliptical clusters.

Feature scaling and PCA: clustering uses distances, so features are standardised (zero mean, unit variance) so that large-valued features such as proline do not dominate. PCA reduces the 13 features to 2 components only for plotting.

Evaluation metrics (no labels needed)

Silhouette score: for a point, $s = (b - a) / \max(a, b)$, where a is the mean distance to its own cluster and b the mean distance to the nearest other cluster. It ranges from −1 to +1; higher is better.

Davies-Bouldin index: the average similarity between each cluster and its most similar cluster (cluster scatter relative to separation). Lower is better; 0 is the minimum.

Calinski-Harabasz index: the ratio of between-cluster to within-cluster dispersion. Higher is better.

Adjusted Rand Index (ARI): an external check that compares the clusters with the known wine cultivars (used only after clustering); 1 means a perfect match, 0 means random.

7. Algorithm / Pseudocode

Step-by-step algorithm for the proposed solution:

```
START

Step 1: Import the libraries and load the Wine dataset; ignore the class labels.
Step 2: Standardise all 13 features with StandardScaler.
Step 3: FOR k = 2 to 8: run K-Means, store inertia and silhouette score.
        Choose k from the elbow of the inertia curve and the highest silhouette score.
Step 4: Fit K-Means (k), Hierarchical Ward (k), DBSCAN (eps, min_samples) and GMM (k).
Step 5: FOR each algorithm: compute silhouette score, Davies-Bouldin index,
        Calinski-Harabasz index and ARI (DBSCAN noise points are excluded from the
scores).
Step 6: Reduce the data to 2 components with PCA and plot every clustering;
        draw the elbow/silhouette curves and the dendrogram.
Step 7: Profile the K-Means clusters (mean of key features) and interpret them.
Step 8: Compare the algorithms and write the conclusion.

STOP
```

08. Technology / Tools / Libraries Used

Technology / Tool	Purpose
Programming Language	Python 3 – implementation of all four clustering algorithms

IDE / Platform	Jupyter Notebook / VS Code – writing, running and visualising the code
Libraries / Packages	scikit-learn (clustering, PCA, metrics), SciPy (dendrogram), NumPy, pandas, Matplotlib (graphs)
Dataset / Other Resources	Wine dataset, built into scikit-learn (178 wine samples, 13 chemical features)

09. Implementation / GitHub Repository

Particular	**Details**
GitHub Repository URL	**https://github.com/Revati-2920/CLUSTERING-ALGORITHMS.git**
Repository Name	**CLUSTERING ALGORITHMS**
Branch Used	**main / master / Other**
Commit ID / Latest Commit	**-**
Date of Last Commit	**25/10/2026**

10. Test Cases / Input and Output

Test Case	Input	Expected Output	Actual Output	Status
TC1: Load dataset	load_wine()	178 samples, 13 features	178 × 13	Pass
TC2: Feature scaling	StandardScaler on the 13 features	Mean ≈ 0, std ≈ 1 for every feature	Mean ≈ 0, std = 1	Pass
TC3: K-Means (k = 3)	Scaled data	Silhouette score > 0	Silhouette = 0.2849	Pass
TC4: Hierarchical (Ward, 3)	Scaled data	Silhouette score > 0	Silhouette = 0.2774	Pass
TC5: DBSCAN (eps 2.3, min_samples 5)	Scaled data	At least 2 clusters found	2 clusters, 42 noise points	Pass
TC6: Gaussian Mixture (3)	Scaled data	Silhouette score > 0	Silhouette = 0.2849	Pass

11. Results / Observations

The Wine dataset has 178 samples and 13 features. After standardisation, K-Means was run for k = 2 to 8. The silhouette score was highest for k = 3 (0.2849) and the elbow of the inertia curve also appears near k = 3, so three clusters were used for K-Means, Hierarchical clustering and GMM. For DBSCAN, eps = 2.3 was chosen close to the median distance to the 5th nearest neighbour (2.22) with min_samples = 5.

Table 1: Clustering quality of the four algorithms

Algorithm	Clusters	Noise points	Silhouette ↑	Davies-Bouldin ↓	Calinski-Harabasz ↑	ARI ↑
K-Means	3	0	0.2849	1.3892	70.94	0.8975
Hierarchical (Ward)	3	0	0.2774	1.4186	67.65	0.7899
DBSCAN	2	42	0.3307	1.1815	70.47	0.3838
Gaussian Mixture	3	0	0.2849	1.3892	70.94	0.8975

Table 2: Profile of the K-Means clusters (mean of original feature values)

Cluster	Alcohol	Flavanoids	Colour intensity	Proline	Size	Interpretation
Cluster 0	12.25	2.05	2.97	510.17	65	Mild, pale wines with low alcohol and low proline
Cluster 1	13.13	0.82	7.23	619.06	51	Very dark (high colour intensity) wines with low flavanoids
Cluster 2	13.68	3.00	5.45	1100.23	62	Strong wines with high alcohol, flavanoids and proline

Output of the program:

```
k vs silhouette: 2: 0.2593, 3: 0.2849, 4: 0.2602, 5: 0.2016, 6: 0.2372, 7: 0.2036, 8: 0.1570  -> best k = 3
K-Means              clusters=3  noise=0  Sil=0.2849  DB=1.3892  CH=70.94  ARI=0.8975
Hierarchical (Ward) clusters=3  noise=0  Sil=0.2774  DB=1.4186  CH=67.65  ARI=0.7899
DBSCAN               clusters=2  noise=42  Sil=0.3307  DB=1.1815  CH=70.47  ARI=0.3838
Gaussian Mixture     clusters=3  noise=0  Sil=0.2849  DB=1.3892  CH=70.94  ARI=0.8975
```

Screenshots / Graphs / Output:

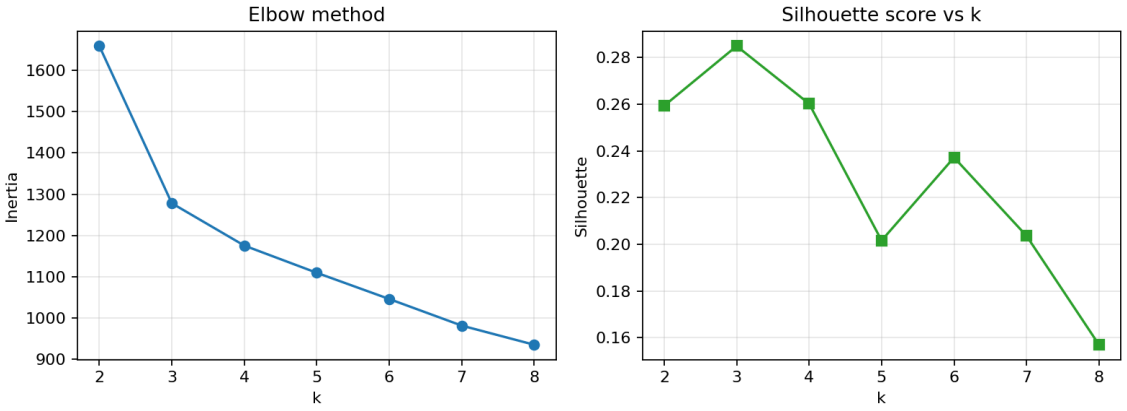


Figure 1: Elbow method and silhouette score for choosing k

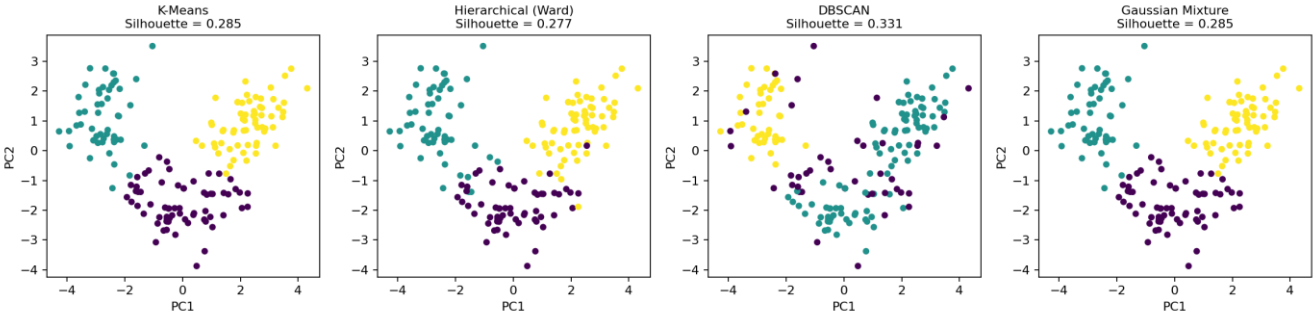


Figure 2: Clusters found by each algorithm, shown on the first two principal components

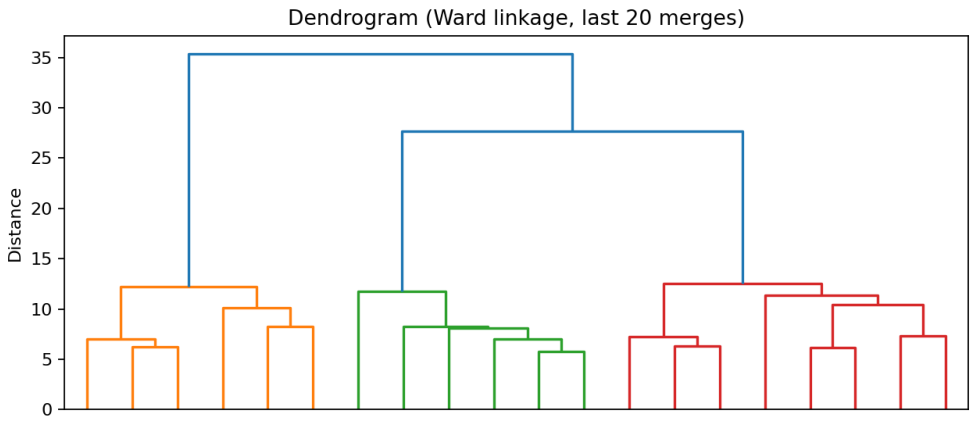


Figure 3: Dendrogram (Ward linkage) – cutting at three clusters gives the three groups

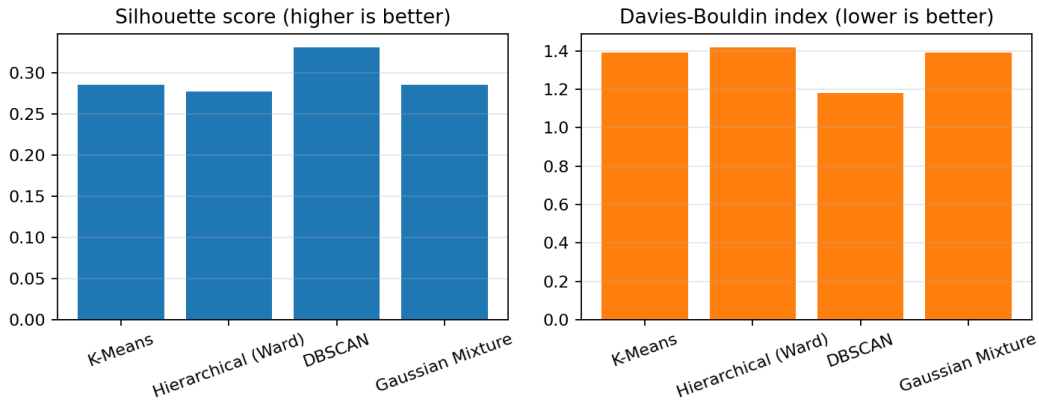


Figure 4: Silhouette score and Davies-Bouldin index of each algorithm

12. Performance Evaluation / Analysis

1. **K-Means** gave a silhouette score of 0.2849 and Davies-Bouldin index of 1.3892 with three clusters of sizes 65, 51 and 62. Its ARI of 0.8975 shows that the groups it found agree closely with the real wine cultivars, even though the labels were never used.
2. **Hierarchical clustering** with Ward linkage produced similar groups (silhouette 0.2774, Davies-Bouldin 1.4186, ARI 0.7899) – slightly lower than K-Means on every measure. Its advantage is the dendrogram, which shows how the clusters merge.
3. **DBSCAN** found 2 dense clusters and labelled 42 of the 178 points (24%) as noise. Its silhouette score (0.3307) and Davies-Bouldin index (1.1815) look best, but this is not a like-for-like comparison: the scores are computed only on the non-noise points and with 2 clusters instead of 3, and its ARI is low (0.3838). The 13-dimensional wine data has no clear density gaps, and the result is very sensitive to eps (for example eps = 2.0 gave 5 clusters and 85 noise points; eps = 2.5 gave 1 cluster and 24 noise points).
4. **Gaussian Mixture** gave exactly the same three clusters as K-Means (identical scores). After scaling, the wine groups are compact and roughly spherical, so the more flexible elliptical model offers no advantage, although it additionally provides membership probabilities.

The silhouette scores (about 0.28) are modest, which means the clusters touch each other: the wine groups overlap somewhat in the 13-dimensional space. Visual inspection (Figure 2) agrees – the three groups are clearly visible on the PCA plane with a small overlap at the boundaries. K-Means gave the best balance of quality, simplicity and agreement with the real cultivars. Because DBSCAN's scores cannot be compared directly, the choice of the final model rests mainly on K-Means, Hierarchical and GMM.

Comparative suitability of the algorithms

Algorithm	Strengths	Limitations	Suitable for
K-Means	Fast and scalable; simple; works well for compact, similar-sized clusters.	k must be given; assumes spherical clusters; sensitive to outliers and scaling.	Customer segmentation, grouping of products or documents, image compression.
Hierarchical	No need to fix k in advance; dendrogram gives a full hierarchy; deterministic.	Slow on large data (at least $O(n^2)$); merges cannot be undone.	Small datasets, taxonomy and gene-expression analysis.
DBSCAN	Finds arbitrary shapes; detects noise and outliers; no k needed.	Sensitive to eps/min_samples; struggles with varying density and high dimensions.	Spatial and geographic data, anomaly and outlier detection.
Gaussian Mixture	Soft assignments; models elliptical clusters of different sizes.	Assumes Gaussian data; can converge to a local optimum; needs the number of components.	Overlapping clusters, probabilistic segmentation, density estimation.

13. Conclusion

K-Means, Hierarchical, DBSCAN and Gaussian Mixture clustering were implemented on the Wine dataset and evaluated with the silhouette score, Davies-Bouldin index, Calinski-Harabasz index and visual inspection. Three meaningful groups of wines were identified – mild pale wines, dark low-flavanoid wines, and strong wines rich in alcohol and proline. K-Means (and the identical GMM) gave the best result among the algorithms that cluster all points (silhouette 0.2849, Davies-Bouldin 1.3892, ARI 0.8975), followed by Hierarchical clustering. It was learnt that feature scaling is essential, that the number of clusters should be chosen with several methods, that no single metric tells the whole story (DBSCAN's scores look good only because noise points are left out), and that the best algorithm depends on the shape, density and size of the data.

14. Assignment Questions

Q1. How was the number of clusters chosen, and what do the silhouette score and Davies-Bouldin index tell you?

Answer: K-Means was run for k = 2 to 8. The silhouette score peaked at k = 3 (0.2849) and the inertia curve bends near k = 3 (elbow), so k = 3 was chosen. A silhouette score near +1 means points are well matched to their own cluster and far from others; the score of about 0.28 indicates moderately separated clusters. The Davies-Bouldin index measures how similar each cluster is to its closest neighbour; lower is better, and 1.39 for K-Means shows some overlap between clusters.

Q2. Compare K-Means, Hierarchical, DBSCAN and GMM. Which is most suitable for this dataset?

Answer: K-Means and GMM gave identical, best-quality results among the algorithms that cluster all points, closely followed by Hierarchical clustering. DBSCAN is unsuitable here because the 13-dimensional data has no clear density gaps, so it labels many points as noise and is very sensitive to eps. K-Means is the most suitable because it is fast, simple and the clusters are compact.

Q3. Why must the data be scaled before clustering, and why is PCA used?

Answer: Clustering depends on distances, so a feature with a large range (such as proline, in the hundreds) would dominate features with small ranges (such as hue). Standardising gives every feature equal weight. PCA is used only to project the 13-dimensional data onto two dimensions so that the clusters can be inspected visually.

15. Implementation / Source Code

Complete, commented source code:

```
"""Practical 5: Clustering on a real-world dataset
Dataset: Wine (chemical analysis of 178 wines, 13 features) - built into scikit-learn.
Algorithms: K-Means, Agglomerative (Ward) hierarchical clustering, DBSCAN, Gaussian Mixture."""
import json
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans, AgglomerativeClustering, DBSCAN
from sklearn.mixture import GaussianMixture
from sklearn.neighbors import NearestNeighbors
from sklearn.metrics import (silhouette_score, davies_bouldin_score,
                             calinski_harabasz_score, adjusted_rand_score)
from scipy.cluster.hierarchy import linkage, dendrogram


# 1. Load and scale the data (labels are NOT used for clustering)
data = load_wine()
X = StandardScaler().fit_transform(data.data)
y_true = data.target                       # used only for an external check at the end
X2 = PCA(n_components=2, random_state=42).fit_transform(X)   # for visualisation only


# 2. Choose the number of clusters (elbow + silhouette)
ks = list(range(2, 9))
inertia, sil = [], []
for k in ks:
    km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X)
    inertia.append(km.inertia_)
    sil.append(silhouette_score(X, km.labels_))
best_k = ks[int(np.argmax(sil))]
print("k vs silhouette:", {k: round(s, 4) for k, s in zip(ks, sil)}, "-> best k =", best_k)


# 3. Fit the four algorithms
K = 3
labels = {
    "K-Means": KMeans(n_clusters=K, n_init=10, random_state=42).fit_predict(X),
    "Hierarchical (Ward)": AgglomerativeClustering(n_clusters=K, linkage="ward").fit_predict(X),
    "DBSCAN": DBSCAN(eps=2.3, min_samples=5).fit_predict(X),
    "Gaussian Mixture": GaussianMixture(n_components=K, random_state=42).fit_predict(X),
}


# 4. Evaluate (internal metrics + adjusted Rand index as an external check)
results = {}
for name, lab in labels.items():
    mask = lab != -1                       # DBSCAN noise points are excluded from the scores
    n_clusters = len(set(lab[mask]))
    results[name] = {
        "clusters": n_clusters,
        "noise": int((~mask).sum()),
        "silhouette": silhouette_score(X[mask], lab[mask]),
        "davies_bouldin": davies_bouldin_score(X[mask], lab[mask]),
        "calinski_harabasz": calinski_harabasz_score(X[mask], lab[mask]),
        "ari": adjusted_rand_score(y_true, lab),
        "sizes": [int((lab == c).sum()) for c in sorted(set(lab))],
    }
    print(name, {k: (round(v, 4) if isinstance(v, float) else v) for k, v in results[name].items()})


# Cluster profile of the K-Means groups (mean of original, unscaled features)
import pandas as pd
```

```
df = pd.DataFrame(data.data, columns=data.feature_names)
df["cluster"] = labels["K-Means"]
key = ["alcohol", "flavanoids", "color_intensity", "proline"]
profile = df.groupby("cluster")[key].mean().round(2)
profile["size"] = df.groupby("cluster").size()
print(profile)

json.dump({"profile": profile.reset_index().values.tolist(), "results": results, "ks": ks, "inertia":
inertia, "sil": sil, "best_k": best_k,
           "n": len(X), "f": X.shape[1]}, open("results.json", "w"), indent=1)

# 5. Plots
fig, ax = plt.subplots(1, 2, figsize=(10, 3.8))
ax[0].plot(ks, inertia, "o-"); ax[0].set_title("Elbow method"); ax[0].set_xlabel("k");
ax[0].set_ylabel("Inertia")
ax[1].plot(ks, sil, "s-", color="tab:green"); ax[1].set_title("Silhouette score vs k");
ax[1].set_xlabel("k"); ax[1].set_ylabel("Silhouette")
for a in ax: a.grid(alpha=.3)
plt.tight_layout(); plt.savefig("elbow_silhouette.png", dpi=160); plt.close()

fig, axes = plt.subplots(1, 4, figsize=(15, 3.8))
for a, (name, lab) in zip(axes, labels.items()):
    a.scatter(X2[:, 0], X2[:, 1], c=lab, cmap="viridis", s=18)
    a.set_title(f"{name}\nSilhouette = {results[name]['silhouette']:.3f}", fontsize=10)
    a.set_xlabel("PC1"); a.set_ylabel("PC2")
plt.tight_layout(); plt.savefig("cluster_plots.png", dpi=160); plt.close()

plt.figure(figsize=(8, 3.6))
dendrogram(linkage(X, "ward"), truncate_mode="lastp", p=20, no_labels=True)
plt.title("Dendrogram (Ward linkage, last 20 merges)"); plt.ylabel("Distance")
plt.tight_layout(); plt.savefig("dendrogram.png", dpi=160); plt.close()

names = list(results)
fig, ax = plt.subplots(1, 2, figsize=(9, 3.6))
ax[0].bar(names, [results[n]["silhouette"] for n in names], color="tab:blue")
ax[0].set_title("Silhouette score (higher is better)")
ax[1].bar(names, [results[n]["davies_bouldin"] for n in names], color="tab:orange")
ax[1].set_title("Davies-Bouldin index (lower is better)")
for a in ax: a.tick_params(axis="x", labelrotation=20); a.grid(axis="y", alpha=.3)
plt.tight_layout(); plt.savefig("metric_comparison.png", dpi=160); plt.close()
```

16. Student Declaration

I hereby declare that the above practical work has been completed by me and that the implementation, analysis, and answers are my own work.

Student Name	PRN	Signature
Revati Bhosale	202401040048	